Arbeitsbericht

Fach: ITP2-I

Name: Mate Szvercseg und Alexander Schauer

Klasse: 4AHITS

Datum: 16.10.2026

Aufgabenstellung: 02_rpi

Display

Aufgabenstellung:
Nimm das angeschlossene Display in Betrieb. Dieses hat die Bezeichnung **0,96 Zoll OLED SSD1306 Display I2C 128 x 64 Pixel**.

Recherchire wie die Pins vom Display mit dem Raspberry angeschlossen werden müssen. Hinweis: Alle Pins sind mit dem Raspberry zu verbinden. Raspberry Doku

Anschlüsse des Displays:

BEACHTE dass die Pins auf der Rückseite herausgeführt sind

Im Raspberry muss das I2C Interface erst aktiviert werden.

Schreibe ein Python Programm um etwas auf dem Display darzustellen.

- Library: adafruit/Adafruit_CircuitPython_SSD1306 Hinweis: es ist unter Umständen nur das Installieren als "virtual environment" möglich. Auf alle Fälle dann wenn die Fehlermeldung error: externally-managed-environment auftritt.

Hinweise zur Library:

- Die Klasse SSD1306_I2C ist von (framebuf.FrameBuffer) abgeleitet. Daher können die Methoden von FrameBuffer verwendet werden (z.B. fill, pixel, ...). D.h. die Beschreibung dieser Befehle findet man in der FrameBuffer Dokumentation.

- Download des zur Textausgabe notwendigen Fonts:

```
$ wget
https://github.com/adafruit/Adafruit_CircuitPython_framebuf/raw/main/examples/font5x8.bin
```

Gib am Display den Hostnamen, die IP Adresse und die Netzwerkmaske (in / Notation) aus:

Hinweis: Keine Werte im Programm vorgeben, d.h. Hostnamen, die IP Adresse und die Netzwerkmaske im Programm ermitteln.

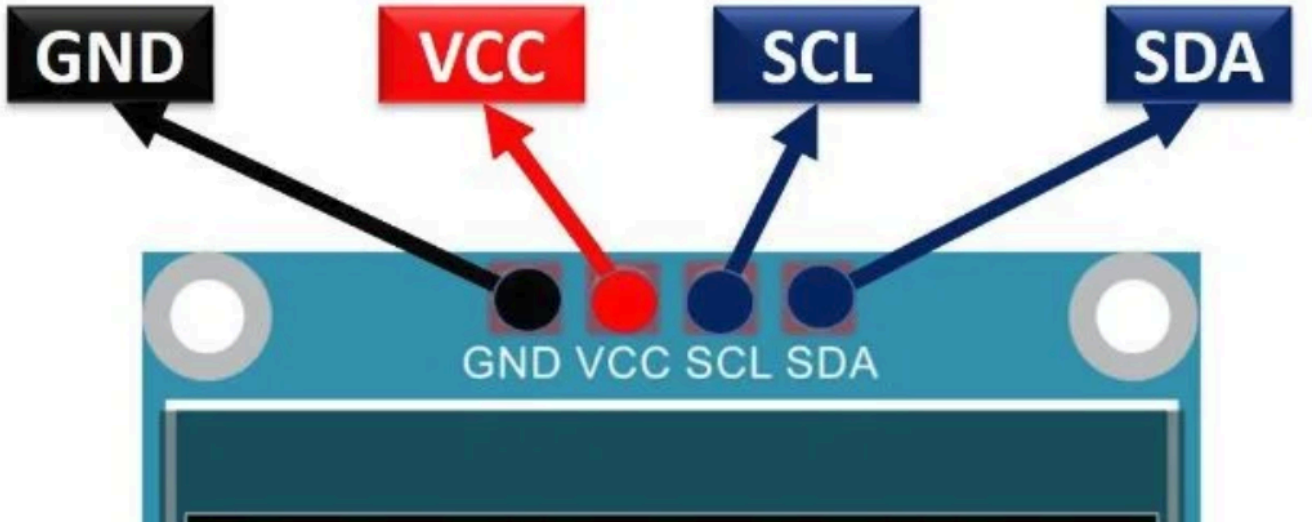
Hinweis 1: Verwende das Python Modul `netifaces`.

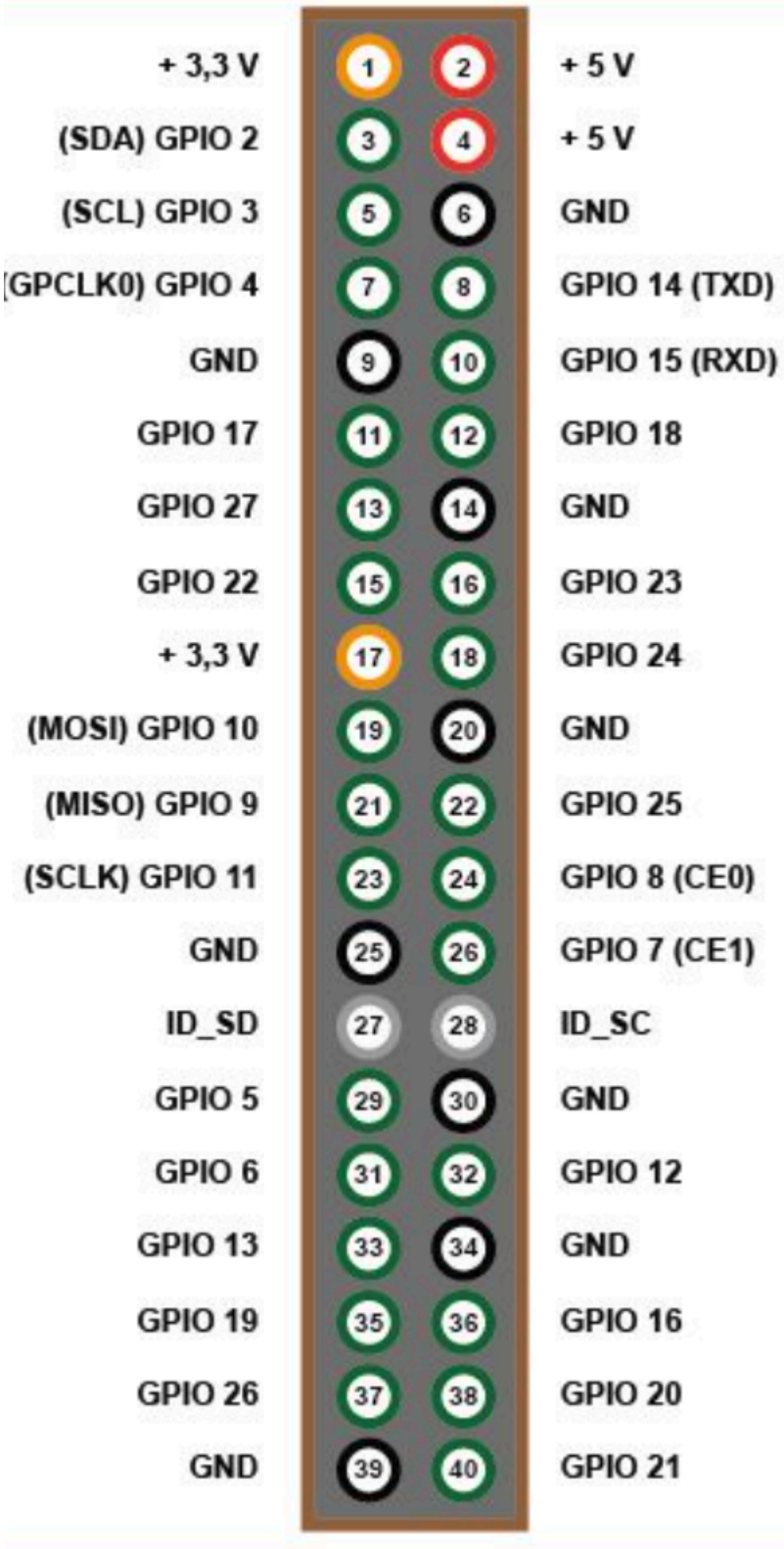
Hinweis 2: Das Modul `ipaddress` ermöglicht aus der Netzwerkmaske die / (Präfix) Notation abzuleiten:

```
import ipaddress
iface=ipaddress.IPv4Interface(('192.168.100.1', '255.255.255.0'))
print(str(iface))
```

Bericht:

Display anstecken:




+ 3,3 V
1
2
+ 5 V
(SDA) GPIO 2
3
4
+ 5 V
(SCL) GPIO 3
5
6
GND
(GPCLK0) GPIO 4
7
8
GPIO 14 (TXD)
GND
9
10
GPIO 15 (RXD)
GPIO 17
11
12
GPIO 18
GPIO 27
13
14
GND
GPIO 22
15
16
GPIO 23
+ 3,3 V
17
18
GPIO 24
(MOSI) GPIO 10
19
20
GND
(MISO) GPIO 9
21
22
GPIO 25
(SCLK) GPIO 11
23
24
GPIO 8 (CE0)
GND
25
26
GPIO 7 (CE1)
ID_SD
27
28
ID_SC
GPIO 5
29
30
GND
GPIO 6
31
32
GPIO 12
GPIO 13
33
34
GND
GPIO 19
35
36
GPIO 16
GPIO 26
37
38
GPIO 20
GND
39
40
GPIO 21

unsere Verkablung:

- GND ist auf Pin 6
- VCC ist auf Pin 4
- SCL ist auf Pin 5
- SDA ist auf Pin 3

I2C Interface aktivieren:

- in terminal `sudo raspi-config` eingeben
- zu Interface Options
- zu I2C
- bei enable auf yes

Libary für Display installieren `--break-system-packages` notwendig sonst error

```
berta@Berta:~$ pip3 install adafruit-circuitpython-ssd1306 --break-system-
packages
Defaulting to user installation because normal site-packages is not
writeable
Looking in indexes: https://pypi.org/simple,
https://www.piwheels.org/simple
Collecting adafruit-circuitpython-ssd1306
  Downloading https://www.piwheels.org/simple/adafruit-circuitpython-
ssd1306/adafruit_circuitpython_ssd1306-2.12.24-py3-none-any.whl.metadata
(5.3 kB)
Collecti
....
```

Fonts installieren:

```
wget
https://github.com/adafruit/Adafruit_CircuitPython_framebuf/raw/main/exampl
es/font5x8.bin
--2026-10-16 13:58:25--
https://github.com/adafruit/Adafruit_CircuitPython_framebuf/raw/main/exampl
es/font5x8.bin
Resolving github.com (github.com)... 20.250.119.67
Connecting to github.com (github.com)|20.250.119.67|:443... connected.
HTTP request sent, awaiting response... 302 Found
Location:
https://raw.githubusercontent.com/adafruit/Adafruit_CircuitPython_framebuf/
main/examples/font5x8.bin [following]
--2026-10-16 13:58:25--
https://raw.githubusercontent.com/adafruit/Adafruit_CircuitPython_framebuf/
main/examples/font5x8.bin
Resolving raw.githubusercontent.com
....
```

Netifaces installieren: (gleiches wie oben mit --break-system-packages)

```
berta@Berta:~ $ pip install netifaces --break-system-packages
Defaulting to user installation because normal site-packages is not
writeable
Looking in indexes: https://pypi.org/simple,
https://www.piwheels.org/simple
Collecting netifaces
  Downloading netifaces-0.11.0.tar.gz (30 kB)
  Preparing metadata
....
```

Ein Python3 Skript Datei erstellen mit Namen display.py

```
import ipaddress
import netifaces
# Bibliotheken importieren
import board
import adafruit_ssd1306
import socket
# I2C-Verbindung zum Display herstellen
i2c = board.I2C()

# Display initialisieren (Breite: 128, Höhe: 64 Pixel)
display = adafruit_ssd1306.SSD1306_I2C(128, 64, i2c)

ipInfo = netifaces.ifaddresses('eth0')[netifaces.AF_INET][0]
ip = ipInfo.get('addr')
subnet = ipInfo.get('netmask')
iface=ipaddress.IPv4Interface((ip,subnet))
hostname = socket.gethostname()
display.text(hostname, 0, 10, 1)
display.text(hostname, 1, 10, 1)
display.text(str(iface), 0, 20, 1)
display.show()
```

Erklärung des Code Snippets:

- adafruit_ssd1306 ist die Libary für Display
- am Anfang wird Verbindung mit Board hergestellt und Display initialisiert
- mit netifaces kann man die IP Adressen der einzelnen Interfaces bekommen (wir brauchen nur Interface eth0)
- die libary ipaddress kann ip Adresse und Subnet verknüpfen
- mit display.text kann man Text ausgeben
- wo die IP adresse ausgegeben wird muss man das iface zu einem String machen
- zum Schluss wichtig `display.show()`

python3 display.py ausführen und STAUNEN

Weitere Aufgabenstellung

Aufgabenstellung:

Das Python Script soll automatisch beim Starten des RPi aufgerufen werden.

Hinweise:

Verwende `rc.local`

Lege im Homverzeichnis des Raspberry Pi User ein shell-script an, dass das python script startet:

```
cd /home/pi/dev
./display_text.py > log.txt 2>&1 &
```

Erklärung:

- Der User heißt pi, das Python Skript muss executable sein und sich im Unterverzeichnis dev befinden.
- Durch > log.txt werden alle entstehenden Ausgaben in die Datei log.txt geschrieben, das ermöglicht eine Fehleranalyse.
- 2>&1 Leitet stderr auf stdout um, sonst würden Fehlerausgaben nicht in die log.txt Datei geschrieben.
- Das & am Ende startet das Python Skript im Hintergrund und setzt die shell script Ausführung fort.
- Das rc.local Skript wird als root ausgeführt, für root können manche Python Module nicht geladen werden. Lösung: mit der sudo Option -u kann die Ausführung als beliebiger User gestartet werden:

`sudo -u pi /home/pi/dev/runrc.sh`

Bericht:

als erstes ein ausführbares Skript erstellen:

```
touch display_booter.sh
chmod +x display_booter.sh
```

Inhalt des Skripts:

```
#!/bin/sh

cd /home/berta
python3 display.py > log.txt 2>&1 &
```

- Shebang Zeile (unser großer Fehler anscheinend ganz wichtig -> siehe Herr Probst Lektion unten)
- in das Berta Home Verzeichnis gehen
- das display.py Skript von vorhin ausführen und Fehler in log.txt speichern

als nächstes dieses Skript beim Starten ausführen indem man in die Datei `/etc/rc.local` ausführbar macht mit `chmod +x` und in diese Datei folgendes schreibt

```
#!/bin/sh

sudo -u berta /home/berta/display_booter.sh
```

- wichtig Shebang Zeile!!!!
- die letzte Zeile führt das Shell Skript von vorhin aus
- `-u berta` führt es nicht als root sondern als berta aus da es sonst Konflikte mit Python Libs geben könnte

Result



Probleme

- SheBang zeile vergessen
- Probleme mit Display Ansteuerung (display.show() vergessen)

Debuggen mit Probst

unser Fragestellung an Herr Probst war wie gefolgt: **"Was macht die Zeile #!/bin/bash?"**

Darauf hin hat Herr Probst uns mit einem Beispiel erklärt was diese Zeile genau macht.

nähmlich Anfang bei der Ausführung des Shellscripts wird mit die Zeile gesagt, dass mit welchen programm der ganze shellscript ausgeführ werden muss (siehe Terminalverlauf)

```
berta@Berta:~ $ vim my_script.sh
berta@Berta:~ $ chmod +ux my_script.sh
chmod: invalid mode: ‘+ux’
Try 'chmod --help' for more information.
berta@Berta:~ $ chmod u+x my_script.sh
berta@Berta:~ $ ./my_script.sh
My Script
berta@Berta:~ $ vim my_ping.shx
berta@Berta:~ $ vim my_ping.shx
berta@Berta:~ $ vim my_ping.sh
berta@Berta:~ $ chmod +x my_ping.sh
berta@Berta:~ $ ./my_ping.sh
/bin/ping: ./my_ping.sh: Name or service not known
berta@Berta:~ $ /bin/ping
/bin/ping: usage error: Destination address required
berta@Berta:~ $ /bin/ping -t 192.168.1.1
/bin/ping: invalid argument: '192.168.1.1'
berta@Berta:~ $ /bin/ping -t
/bin/ping: option requires an argument -- 't'

Usage
  ping [options] <destination>

Options:
<destination>      DNS name or IP address
  -3                 RTT precision (do not round up the result time)
  -a                 use audible ping
  -A                 use adaptive ping
  -B                 sticky source address
  -c <count>         stop after <count> replies
  -C                 call connect() syscall on socket creation
  -D                 print timestamps
  -d                 use SO_DEBUG socket option
  -e <identifier>    define identifier for ping session, default is random
for
                     SOCK_RAW and kernel defined for SOCK_DGRAM
                     Imply using SOCK_RAW (for IPv4 only for identifier 0)
  -f                 flood ping
  -h                 print help and exit
  -H                 force reverse DNS name resolution (useful for numeric
                     destinations or for -f), override -n
  -I <interface>     either interface name or address
  -i <interval>      seconds between sending each packet
  -L                 suppress loopback of multicast packets
```

```
  -l <preload>       send <preload> number of packages while waiting
replies
  -m <mark>          tag the packets going out
  -M <pmtud opt>     define path MTU discovery, can be one of
<do|dont|want|probe>
  -n                 no reverse DNS name resolution, override -H
  -O                 report outstanding replies
  -p <pattern>       contents of padding byte
  -q                 quiet output
  -Q <tclass>        use quality of service <tclass> bits
  -s <size>          use <size> as number of data bytes to be sent
  -S <size>          use <size> as SO_SNDBUF socket option value
  -t <ttl>           define time to live
  -U                 print user-to-user latency
  -v                 verbose output
  -V                 print version and exit
  -w <deadline>      reply wait <deadline> in seconds
  -W <timeout>       time to wait for response

IPv4 options:
  -4                 use IPv4
  -b                 allow pinging broadcast
  -R                 record route
  -T <timestamp>     define timestamp, can be one of
<tsonly|tsandaddr|tsprespec>

IPv6 options:
  -6                 use IPv6
  -F <flowlabel>     define flow label, default is random
  -N <nodeinfo opt>  use IPv6 node info query, try <help> as argument

For more details see ping(8).
berta@Berta:~ $ /bin/ping 192.168.1.1
PING 192.168.1.1 (192.168.1.1) 56(84) bytes of data.
^C
--- 192.168.1.1 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3059ms

berta@Berta:~ $ /bin/ping 192.168.15.1
PING 192.168.15.1 (192.168.15.1) 56(84) bytes of data.
^C
--- 192.168.15.1 ping statistics ---
3 packets transmitted, 0 received, 100% packet loss, time 2045ms

berta@Berta:~ $ /bin/ping 192.168.15.51
PING 192.168.15.51 (192.168.15.51) 56(84) bytes of data.
64 bytes from 192.168.15.51: icmp_seq=1 ttl=64 time=0.221 ms
64 bytes from 192.168.15.51: icmp_seq=2 ttl=64 time=0.144 ms
64 bytes from 192.168.15.51: icmp_seq=3 ttl=64 time=0.150 ms
^C
--- 192.168.15.51 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2042ms
rtt min/avg/max/mdev = 0.144/0.171/0.221/0.034 ms
berta@Berta:~ $ vim my_ping.sh
```

```
berta@Berta:~ $ vim my_ping.sh
berta@Berta:~ $ ./my_ping.sh
/bin/ping: ./my_ping.sh: Name or service not known
berta@Berta:~ $ vim my_ping.sh
berta@Berta:~ $ ./my_ping.sh
/bin/ping: ./my_ping.sh: Name or service not known
berta@Berta:~ $ /bin/ping
/bin/ping: usage error: Destination address required
berta@Berta:~ $ /bin/ping 192.168.15.51
PING 192.168.15.51 (192.168.15.51) 56(84) bytes of data.
64 bytes from 192.168.15.51: icmp_seq=1 ttl=64 time=0.163 ms
64 bytes from 192.168.15.51: icmp_seq=2 ttl=64 time=0.128 ms
^C
--- 192.168.15.51 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1028ms
rtt min/avg/max/mdev = 0.128/0.145/0.163/0.017 ms
berta@Berta:~ $ ./my_ping.sh
/bin/ping: ./my_ping.sh: Name or service not known
berta@Berta:~ $ ^C
berta@Berta:~ $ vim my_ping.sh
berta@Berta:~ $ ./my_ping.sh
/bin/ping: ./my_ping.sh: Name or service not known
berta@Berta:~ $ vim my_ping.sh
berta@Berta:~ $ vim my_ping.sh
berta@Berta:~ $ cat my_ping.sh

berta@Berta:~ $ cat my_ping.sh
#!/bin/ping
192.168.15.51
berta@Berta:~ $ cat my_script.sh
#!/bin/bash
echo "My Script"
berta@Berta:~ $
```